# MOBILE DEVELOPMENT

## Android / iOS with Flutter

**Renel Sainfort**
ESILV – Engineering School

### CM5: Testing - Security - Deployment

# SESSION

01 — Recap CM4
02 — Introduction
03 — Testing
04 — Security
05 — Deployment
06 — Extras : Sub-agents & Skills
07 — What's next?

# CM4 Retrospective

# Introduction

### Testing

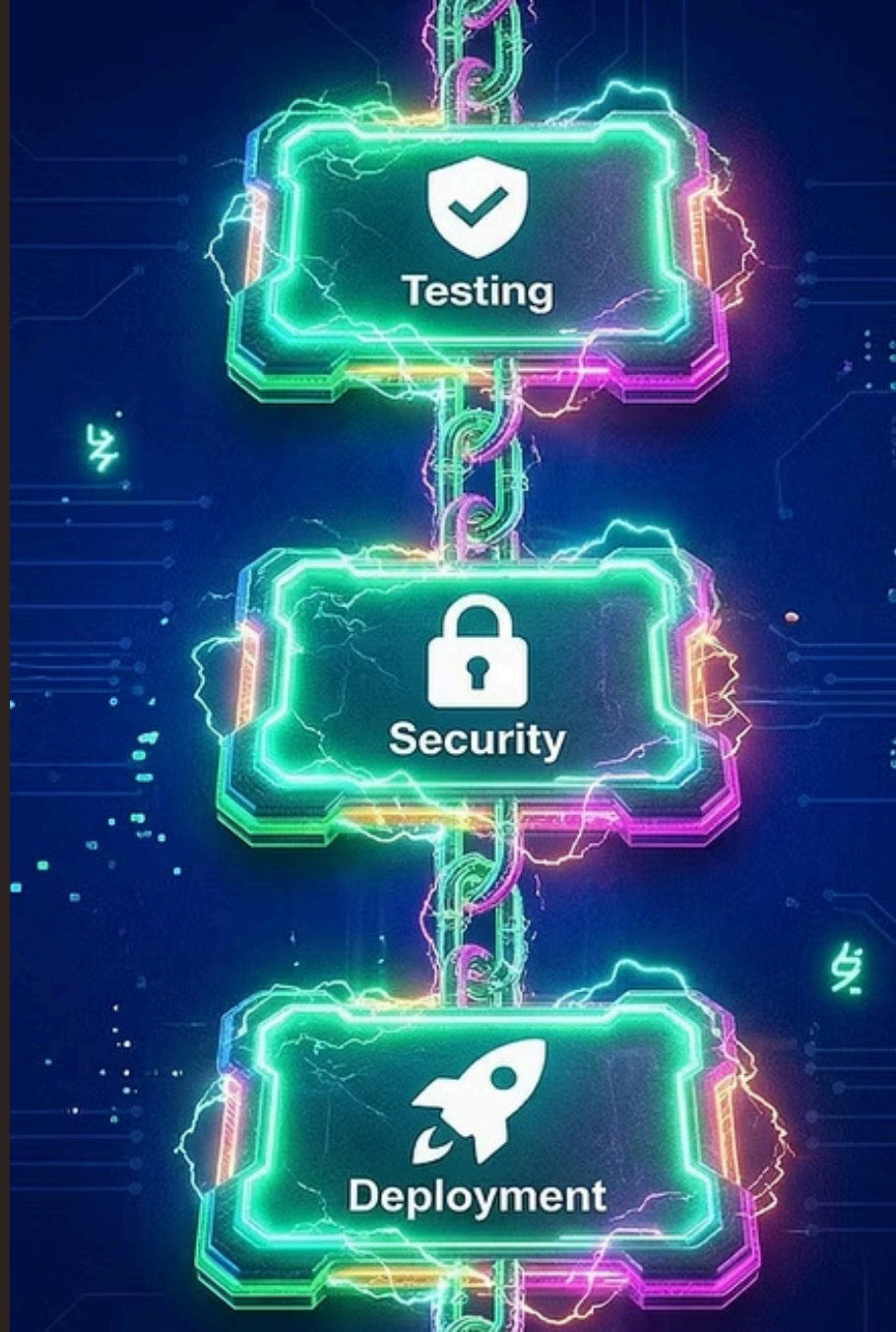
Does it work as expected?

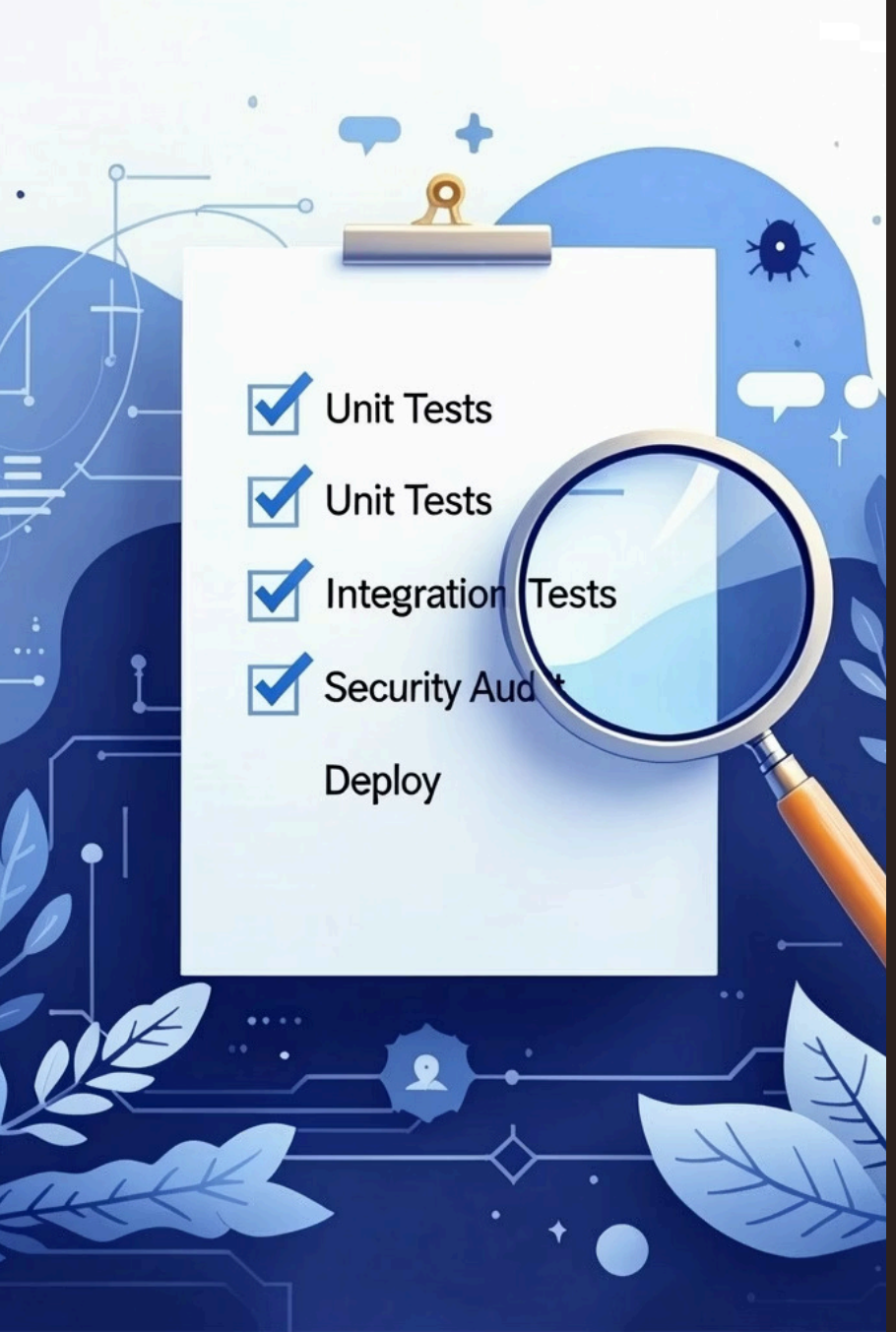
### Security

Is it safe for users?

### Deployment

Is it live & maintainable?

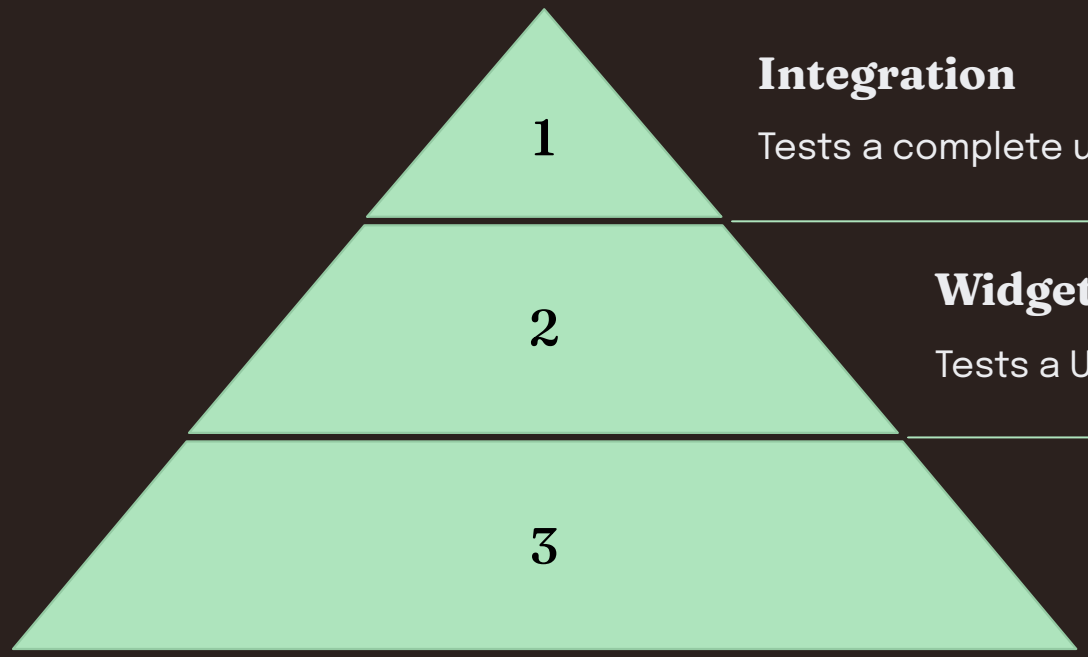





Chapter 03

# Testing

# Testing — The 3 Levels




**Integration**

Tests a complete user scenario end to end

**Widget**

Tests a UI component behavior

**Unit**

Tests a single function in isolation

# Testing by Architecture Layer

| Layer | Test Type | What We Test |
|---|---|---|
| Model | Unit Test | Pure business logic |
| Repository | Unit Test + Mock | Data sources |
| Service | Unit Test | Business use cases |
| Controller | Unit Test | Riverpod state |
| Screen | Widget Test | UI behavior |
| Full App | Integration Test | User journey |

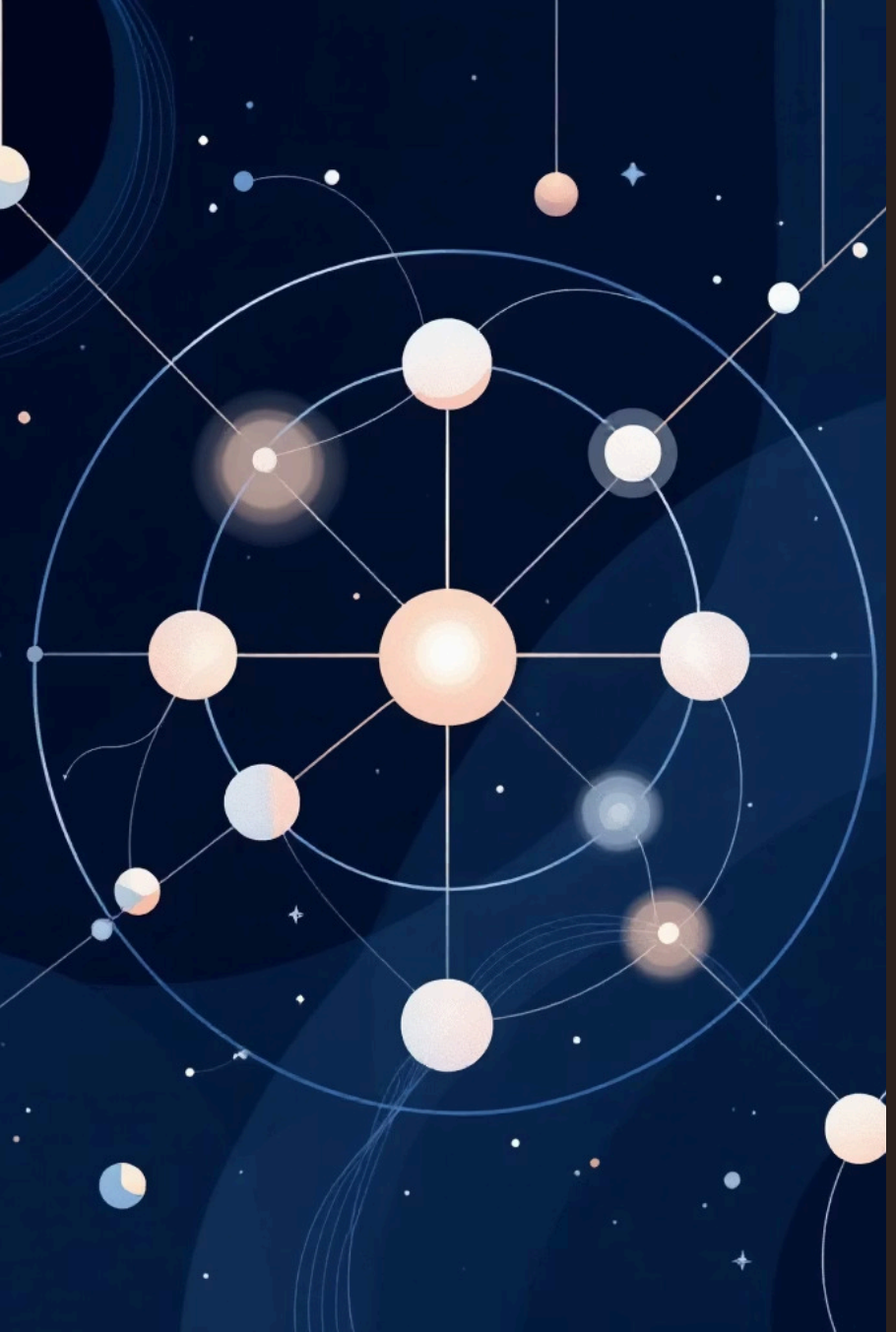
# TDD — Test Driven Development

*"You write the test before writing the feature – the test defines what the code must do."*

1. **RED**
   Write a test that fails
2. **GREEN**
   Write the minimum code to pass it
3. **REFACTOR**
   Clean the code

# Testing tools

| | Unit Test | Widget Test | Integration Test |
|---|---|---|---|
| **Tool** | flutter_test + mockito | testWidgets | Patrol / integration_test |



CHAPTER 04

# Security

# Security Levels

| Level | Problem | Concrete Example |
|---|---|---|
| **Code** | API Key Exposure | Supabase key hardcoded in main.dart pushed to GitHub |
| **Data** | Data Leak | Supabase RLS not configured – all users see all data |
| **Network** | Injection | User input passed directly into a SQL query or an AI prompt |
| **User** | Multi-tenant / Auth | JWT token still valid after account deletion |

# Security Tools

| Level | Tool | Role |
|---|---|---|
| **Code** | flutter_dotenv + .gitignore | Store keys outside the codebase |
| **Data** | Supabase RLS / Firestore Rules | Filter data per user |
| **Network** | Supabase SDK (prepared queries) + flutter_form_builder | Validate and sanitize user inputs |
| **User** | Supabase Auth / Firebase Auth | Manage sessions, tokens, multi-tenant access |

## AI Audit – Claude Code / Gemini CLI can automatically scan your code and detect :

- API keys exposed in the code
- Missing RLS rules
- Unvalidated user inputs

# Security Standards & References

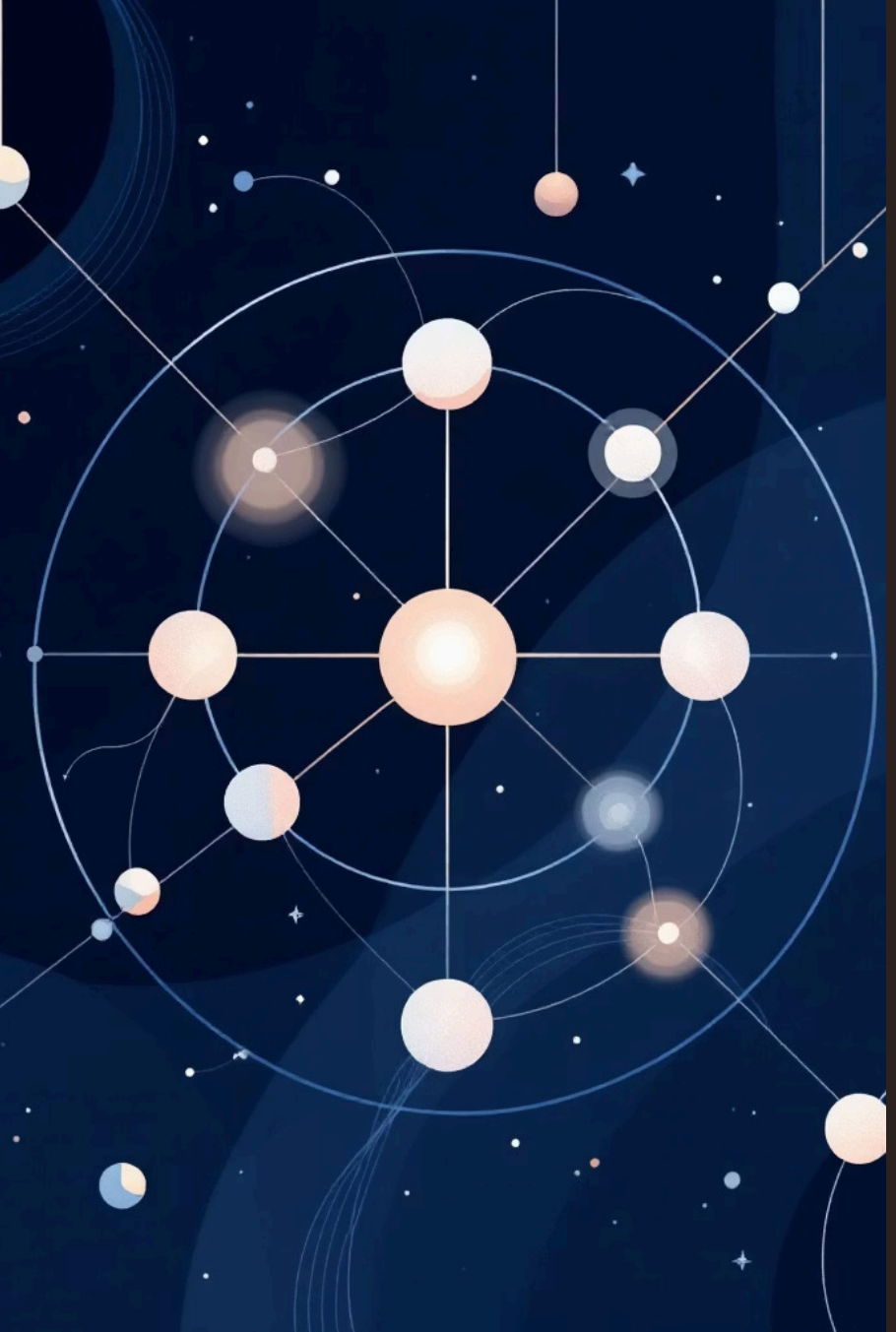
### OWASP

Open Web Application Security Project – worldwide reference on application security. Publishes the annual Top 10 critical vulnerabilities. Free & open source.

### OWASP MAS

Mobile Application Security – mobile-specific version. Contains the MASVS (Mobile Application Security Verification Standard) for iOS & Android apps.

🔗 **owasp.org**



CHAPTER 05

# Deployment

# Google Play — Deployment Process

## Prerequisites

Google Play Developer account - One-time fee: $25

## Steps

- Configure signing in android/app/build.gradle
- flutter build appbundle --release → generates a signed AAB
- Upload the AAB to Play Console
- Choose a track → Internal Testing / Production

## Tracks

**Internal Testing**

Max 100 testers – available immediately

**Production**

Google review – available within 24-48h

# App Store — Deployment Process

## Prerequisites

Apple Developer account – Annual fee: $99/year

## Steps

- Create the app in App Store Connect
- Configure signing in Xcode — certificates + provisioning profiles
- flutter build ipa --release → generates a signed IPA
- Upload via Xcode or Transporter
- Choose a destination → TestFlight / App Store

## Tracks

**TestFlight**

Max 10,000 testers – available in less than 1 hour

**Production**

Apple review, 1 to 3 days – most rejections due to metadata, not code

# Flutter Web — Deployment Process

## Prerequisites

- A web hosting service

## First Deployment

- Enable web support: flutter config --enable-web
- Build the web app: flutter build web
- Deploy the build/web folder to your hosting
- Available immediately — no store, no review

## Hosting Options

**Firebase Hosting**

Free, fast, Google CDN

**Vercel**

Free, native GitHub integration

**VPS (Hostinger)**

Full control, Dokploy integration

## Every Update

Same process – rebuild and redeploy. Update is live instantly.

# Choosing Your Deployment Target

*The right platform depends on your project stage*

1

**PROTOTYPE**

Flutter Web

*Show fast / free*

2

**MVP**

Only Google Play

*Test market*

3

**CLIENT**

App Store + Google Play

*Ship to client*

# The DevOps Role

*DevOps engineers bridge the gap between development and production – they automate, secure and deliver software continuously.*

1

### Build Automation — CI

Automate the app compilation process using CI pipelines

2

### Testing Automation — CI

Trigger automated tests on every code change to catch bugs early

3

### Security — CI

Scan and protect the codebase from vulnerabilities

4

### Deployment — CD

Deliver the app to stores or hosting environments automatically

# DevOps in 2026

*Small agencies can now build their own pipeline – without a dedicated DevOps engineer.*

1. **git push → main**
   Triggers GitHub Actions

2. **Testing — CI**
   flutter test · AI generates tests

3. **Security — CI**
   flutter analyze · dart pub audit · AI security audit

4. **Deployment — CD**
   Fastlane → Google Play / App Store · Dokploy → Web / VPS

CHAPTER 06

# Extras :

# Sub-agents & Skills

# Sub-agents

*A dedicated AI agent with a single responsibility.*

A sub-agent is an AI instance configured to handle one specific task – it doesn't do everything, it does one thing well.

## Use Case — Security Sub-agent

### Scan API keys

Detects exposed keys in the codebase

### Check RLS rules

Validates Supabase row-level security

### Validate user inputs

Ensures inputs are sanitized and safe

### Generate security report

Produces a structured audit summary

*"One agent. One responsibility. Always available."*

# Skills

*Reusable capabilities you give to an agent.*

A skill is a packaged action – like an alias – that an agent can call to accomplish a specific task.

Need Instructions - Outils - Contexte

## Two Types

**Native**

Provided by the platform – Word, PDF, PowerPoint

**Custom**

Created by you for your specific workflow

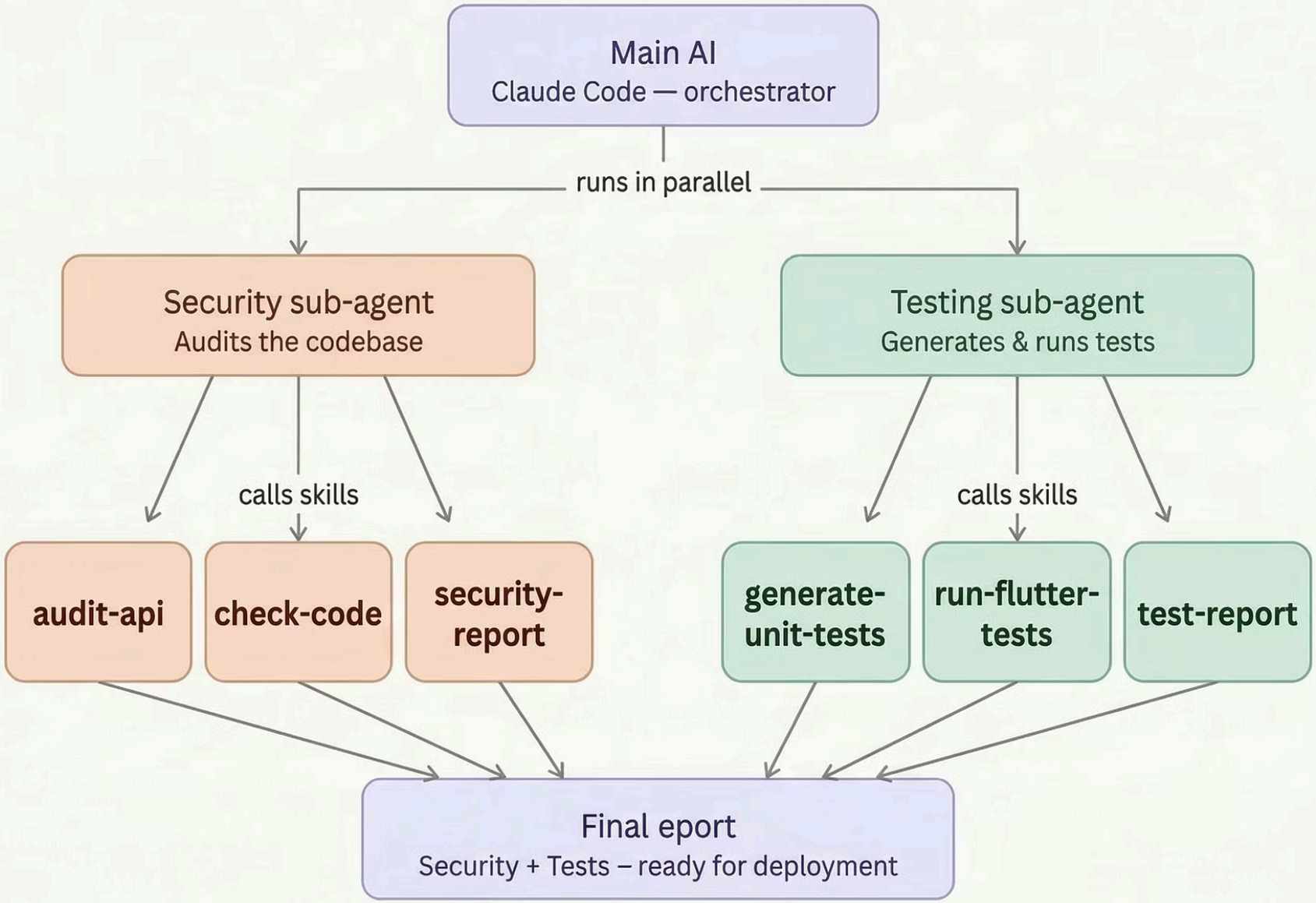
## Use Case — Security Sub-agent

1 **audit-api-keys**

2 **check-rls-rules**

3 **generate-security-report**

*"Build once. Reuse everywhere."*




Skills context : CLAUDE.md + OWASP M/S + project architecture

# WHAT'S NEXT

**TP5**

Continue your project

CM6

Final CM